

CIPHERSHIELD

DATA ENCRYPTION & DECRYPTION TOOL

A Python-Based Cryptographic Application for Secure Message and File Protection

Name	Radifa Khanam
Project	Data Encryption and Decryption Tool
Environment	Python 3.14.7 on Kali Linux
Technologies	AES-256-GCM, RSA-2048-OAEP, Tkinter

1. Executive Summary

CipherShield is a Python-based cryptographic application developed to demonstrate secure data protection techniques. The application implements modern encryption algorithms including AES-256-GCM and RSA-2048-OAEP for protecting messages and files. It provides encryption, decryption, integrity verification, file security features, and a graphical user interface for practical usage.

2. Introduction and Background

Digital information is continuously exposed to security threats such as unauthorized access, data leakage, and interception. Encryption converts readable information into protected ciphertext so that only authorized users with valid keys can recover the original information. This project demonstrates practical implementation of cryptographic principles including confidentiality, integrity, and authentication.

3. Problem Statement

Sensitive information stored or transmitted without protection can be accessed by unauthorized users. A secure encryption application is required to demonstrate how modern cryptographic algorithms can protect messages and files while maintaining data integrity.

4. Objectives and Scope

The objectives of CipherShield are to implement secure encryption mechanisms, provide message and file protection, demonstrate authenticated encryption, compare encryption performance, and create an easy-to-use graphical interface.

5. Requirements Analysis

Functional requirements include message encryption, message decryption, file encryption, file decryption, secure key generation, tamper detection, and performance testing. Non-functional requirements include usability, reliability, security, and maintainable modular implementation.

6. Tools and Technologies

Technology	Purpose
Python 3.14.7	Core application development
Cryptography Library	Secure cryptographic operations
Tkinter	Graphical user interface
AES-256-GCM	Symmetric authenticated encryption
RSA-2048-OAEP	Asymmetric encryption
Kali Linux	Development and testing environment

7. System Architecture

The application follows a modular architecture where user input is processed through the GUI and passed to dedicated encryption modules.

User Input → CipherShield GUI → AES/RSA Encryption Engine → Security Validation → Encrypted Output

8. Functional Modules

- AES-256-GCM Encryption Module: Provides authenticated symmetric encryption.
- AES-256-GCM Decryption Module: Validates and recovers original information.
- RSA-2048-OAEP Module: Demonstrates public-key and private-key encryption.
- File Security Module: Encrypts and decrypts files using AES protection.
- Tamper Detection Module: Rejects modified encrypted data.
- Performance Module: Measures AES and RSA execution time.

9. Cryptographic Implementation

AES-256-GCM uses a 256-bit secret key and authenticated encryption. It provides confidentiality and integrity through encryption and authentication verification.

RSA-2048-OAEP uses asymmetric cryptography where the public key performs encryption and the private key performs decryption.

10. Testing and Results

Test Case	Status
AES message encryption/decryption	Pass
RSA message encryption/decryption	Pass
File encryption/decryption	Pass
AES-GCM tamper detection	Pass
Performance comparison	Completed

11. Performance Analysis

Testing environment: Kali Linux, Python 3.14.7, Cryptography Library 50.0.1.

Algorithm	Average Encryption Time
AES-256-GCM	0.001179 ms
RSA-2048-OAEP	0.040221 ms

The measured results show AES-256-GCM achieved lower average encryption time in this test environment. Results may vary depending on hardware and workload.

12. Project Evidence - Screenshots

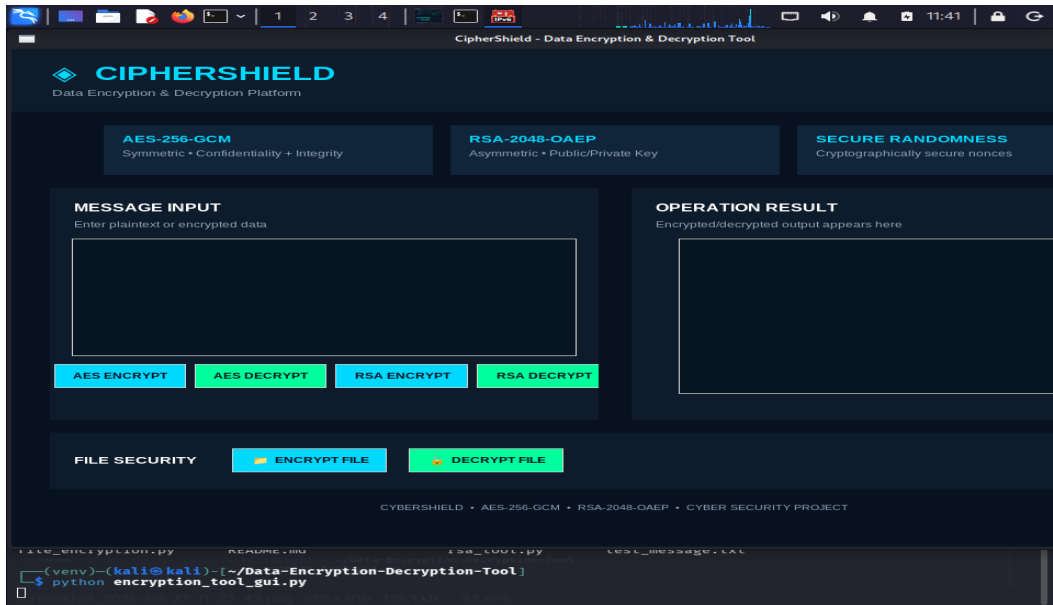


Figure 1: CipherShield Main GUI

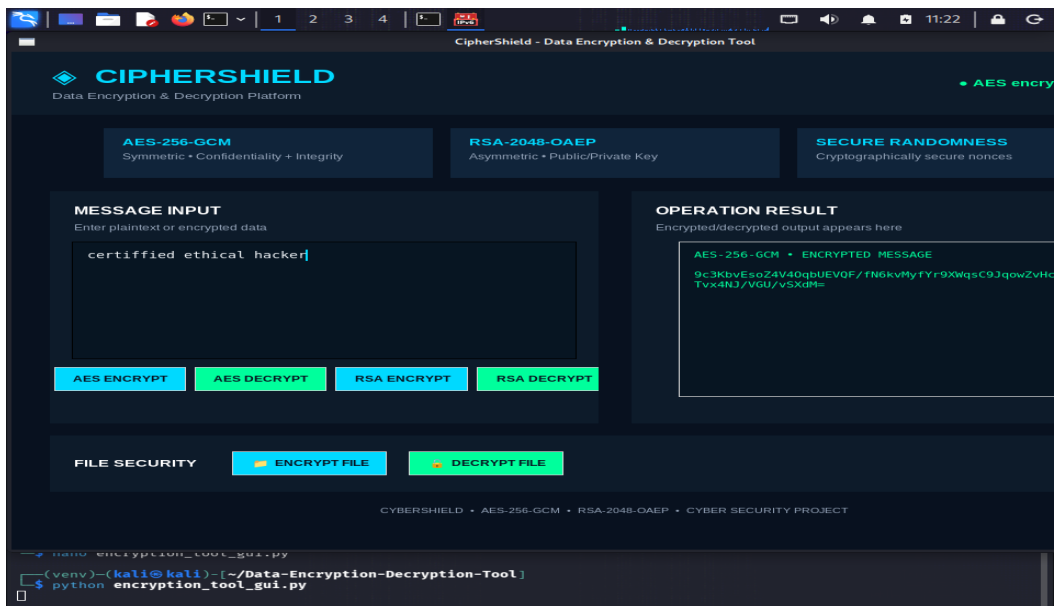


Figure 2: AES Encryption Output

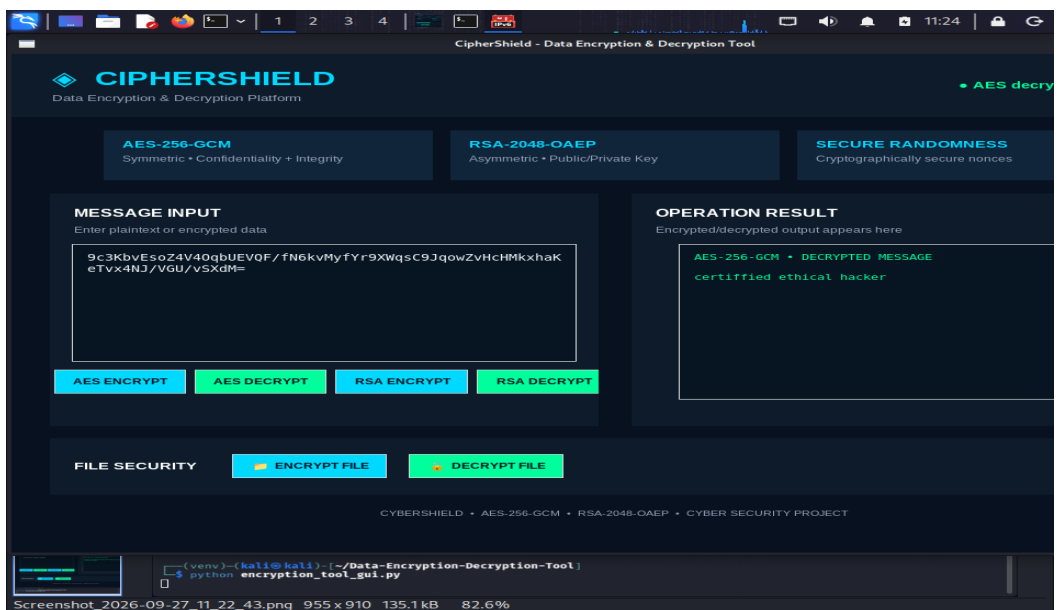


Figure 3: AES Decryption Output

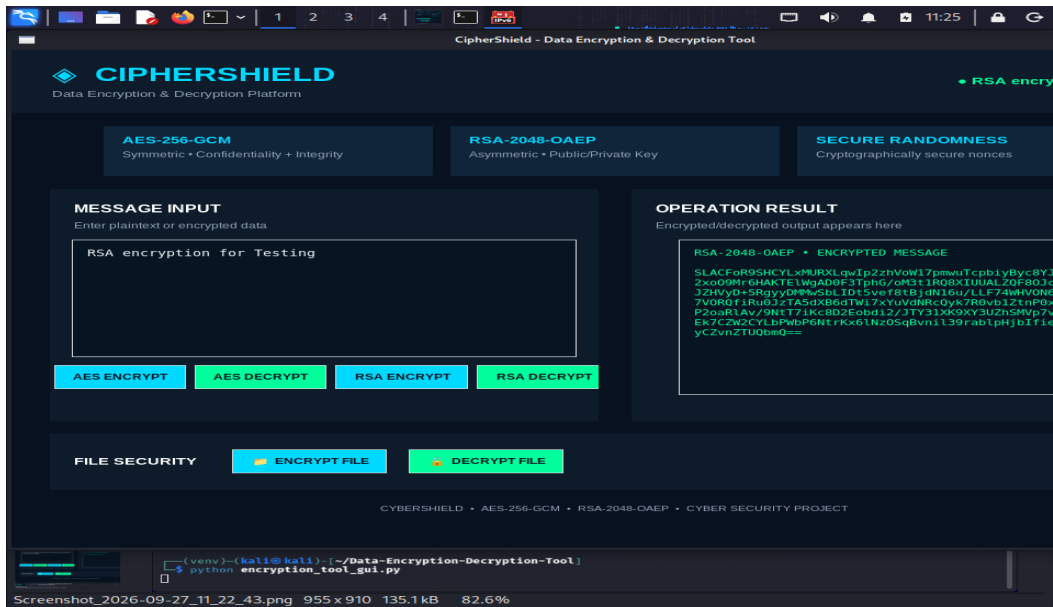


Figure 4: RSA Encryption Output

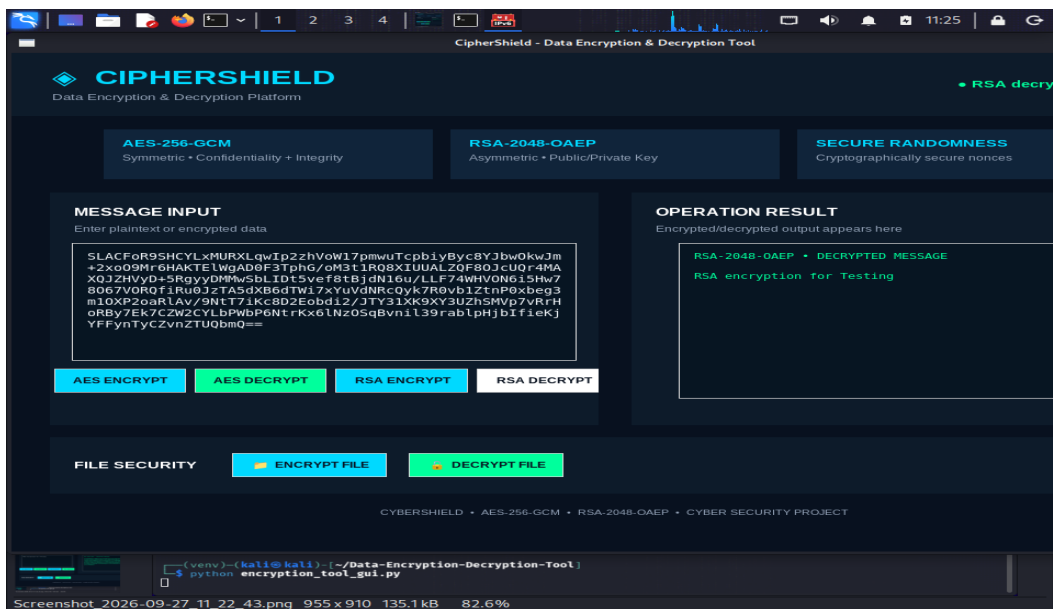


Figure 5: RSA Decryption Output

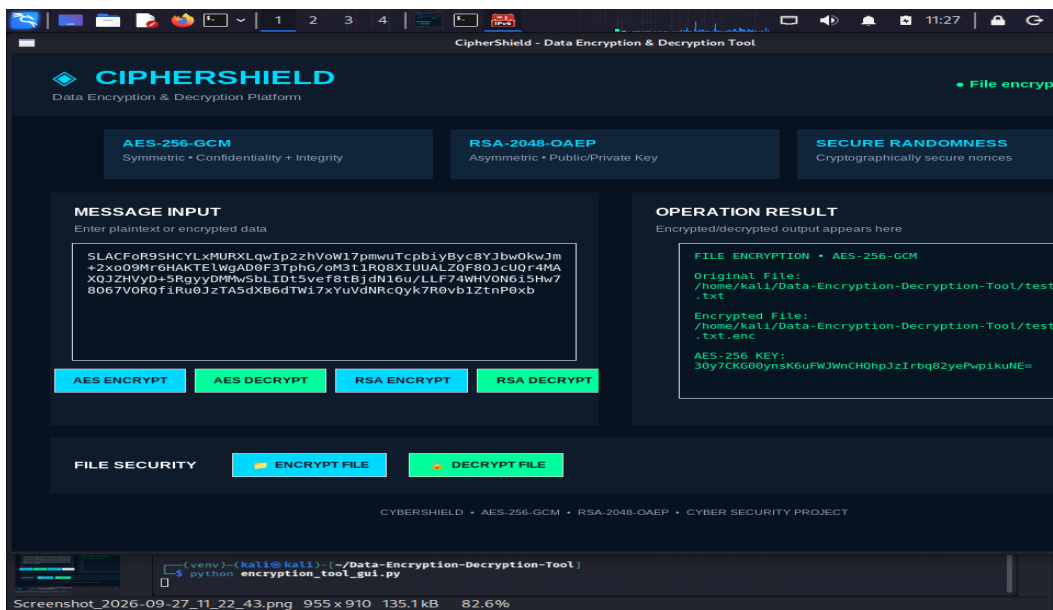


Figure 6: File Encryption

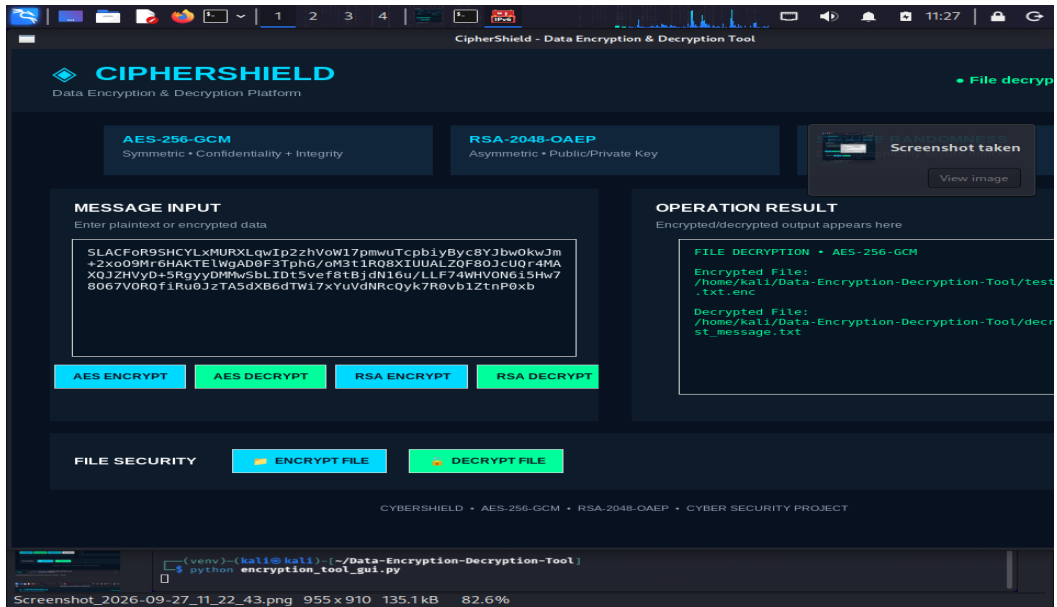


Figure 7: File Decryption

13. Security Considerations

The project uses authenticated encryption, secure cryptographic libraries, random nonce generation, and integrity validation. Sensitive keys should always be protected using secure key management practices in production environments.

14. Limitations

- Key management is designed for demonstration purposes.
- No enterprise authentication system is integrated.
- The project focuses on educational cryptographic implementation.

15. Future Enhancements

- Hybrid AES-RSA encryption workflow.
- Digital signature implementation.
- Secure key storage and rotation.
- User authentication and access control.
- Security logging and auditing.

16. Conclusion

CipherShield successfully demonstrates the practical implementation of modern cryptographic concepts through a Python-based encryption and decryption platform. The project integrates AES-256-GCM for authenticated symmetric encryption, RSA-2048-OAEP for asymmetric encryption, secure file protection, tamper detection, performance evaluation, and a graphical interface within a single application. The completed tests verified successful message encryption and decryption, RSA encryption and recovery, file encryption and restoration, and rejection of modified authenticated data. The performance experiment also provided a practical comparison of the measured execution times of AES-256-GCM and RSA-2048-OAEP in the selected test environment. Overall, the project provides a strong practical foundation for understanding confidentiality, integrity, authentication, key usage, and the role of cryptographic algorithms in secure data protection. Its modular structure also provides a basis for future improvements such as stronger key management, hybrid encryption, digital signatures, user authentication, audit logging, and other production-oriented security controls.

17. References

- 1. NIST, FIPS 197, Advanced Encryption Standard (AES).
- 2. NIST, SP 800-38D, Recommendation for Block Cipher Modes of Operation: GCM and GMAC.
- 3. NIST, SP 800-57 Part 1 Rev. 5, Recommendation for Key Management.
- 4. RFC 8017, PKCS #1: RSA Cryptography Specifications Version 2.2.
- 5. Python Cryptography Documentation, cryptography.io.
- 6. OWASP, Cryptographic Storage Cheat Sheet.
- 7. Python Documentation, secrets module and secure random generation.
- 8. Python Documentation, tkinter GUI toolkit.
- 9. NIST, SP 800-175B, Guideline for Using Cryptographic Standards.
- 10. NIST Cybersecurity Framework documentation.
- 11. Kali Linux Documentation.
- 12. PyCA cryptography project documentation.